Year: 2026-2027 **Sem V-NEP**

Course: Agile Software Development & DevOps Lab

Class: TE (T2) **Date of Submission: 15/10/2026**

A Mini Project Report on

FoodFlow

A Food Delivery Application Delivered Through an Agile and DevOps Pipeline

Submitted in partial fulfilment of the requirements of the course

Agile Software Development & DevOps Lab

Semester V (NEP) · Academic Year 2026–2027 · Class TE (T2)

Submitted by

Name	Roll No.
Bhoomi Sakhrani	2401097
Ekta Sawant	2401103

Under the guidance of

Prof. Sanober Shaikh

Department of Artificial Intelligence & Data Science

Thadomal Shahani College of Engineering

2026–2027

CERTIFICATE

This is to certify that the mini project entitled “FoodFlow: A Food Delivery Application Delivered Through an Agile and DevOps Pipeline” is a bona fide work of Bhoomi Sakhrani (2401097) and Ekta Sawant (2401103), submitted in partial fulfilment of the requirements of the course Agile Software Development & DevOps Lab, Semester V, during the academic year 2026–2027.

Prof. Sanober Shaikh Project Guide	Dr. Madhuri Rao Head of Department

Date: 15/10/2026 Place: Mumbai

ABSTRACT

Ordering food by phone or by walking into a restaurant is slow, error-prone and gives the customer no visibility of what happens after the order is placed. FoodFlow addresses this with a web application in which a customer browses restaurants and their menus, places an order, and follows it in real time from the kitchen to the doorstep. Restaurant staff work a live order queue and manage their menu, and delivery partners pick up ready orders and update the delivery status.

The application was used as the vehicle for applying the practices of the Agile Software Development & DevOps Lab to a single product. Work was planned in Jira as a Scrum backlog of fourteen issues across two sprints, each issue developed on its own Git branch and merged into main through a GitHub pull request. Every component is containerised with Docker and composed with MongoDB, Prometheus and Grafana into a five-service Docker Compose stack. A GitHub Actions workflow installs dependencies, lints, tests and builds every branch, and on main additionally deploys the stack, smoke-tests the exact version that was built, and rolls back to the last good version on failure. Prometheus records business and

Keywords: Agile, Scrum, Jira, Git, GitHub, Docker, Docker Compose, GitHub Actions, CI/CD, Prometheus, Grafana, React, Node.js

CONTENTS

Chapter	Title
1	Introduction
2	Requirements and Agile Planning (Experiment 6)
3	System Design
4	Application Implementation
5	DevOps Implementation (Experiments 1–7)
6	Testing
7	Results and Discussion
8	Conclusion and Future Scope

1. Introduction

1.1 Background

Demand for food delivery arrives in sharp peaks: within a short window around lunch and dinner, a restaurant receives more orders than its kitchen can prepare, and customers who order by phone or message have no reliable way to know whether the order was understood, when it will be ready or where the delivery partner is. The result is wrong orders, uneven kitchen workload, idle delivery partners and no record of what was sold or when demand peaks.

The Agile Software Development & DevOps Lab introduced, through separate experiments, the practices used to deliver software of this kind reliably: version control, containers, multi-container orchestration, continuous integration, automated deployment, Agile project tracking and monitoring. Each experiment was performed in isolation on a toy workload. This mini project applies them to one realistic product so that their interaction—rather than each tool on its own—becomes visible.

1.2 Problem Statement

To design, build and operate a web application that lets customers order food from restaurants and track the delivery, and to deliver it through an Agile, fully automated pipeline in which every change is planned as a Jira issue, versioned in Git, tested and built by GitHub Actions, deployed as containers and observed through Prometheus and Grafana.

1.3 Objectives

1. Allow customers to register, browse restaurants and menus, place an order and track it live until it is delivered.
2. Give restaurant staff a live queue in which orders move through a fixed sequence of states, and allow dishes to be marked unavailable instantly.
3. Give delivery partners a list of ready orders to pick up and a way to report the delivery status.
4. Plan the work as a Scrum backlog in Jira with story points, sprints and a Definition of Done linked to the build pipeline.
5. Version every change on a branch named after its Jira issue and merge only through pull requests.
6. Containerise every component and run the full stack with one Docker Compose command.
7. Build, lint and test every branch automatically with GitHub Actions, and deploy main automatically with a smoke test and rollback.
8. Monitor business and service metrics over time with Prometheus and Grafana.

1.4 Scope

The system targets a set of restaurants served from a single platform. Payment is out of scope: orders are paid in cash on delivery. Notifications are delivered through the live tracking page rather than SMS or push messages. Delivery partners are shown the orders that are ready and accept one at a time; route optimisation and live GPS tracking are not attempted. The deployment target is a single host running Docker; the same Compose file and pipeline would apply to a cloud server.

1.5 Mapping of Lab Experiments to the Project

Exp	Lab topic	Application in FoodFlow	Section
1	Version control with Git	Branch per Jira issue, 14 pull requests, merge commits, release tag v1.0	5.1
2	Containerisation with Docker	Dockerfiles for the API (non-root, health check) and the frontend (multi-stage, Nginx)	5.2
3	Docker Compose	Five-service stack with health-gated start-up, named volume, one network	5.3
4	Continuous integration	GitHub Actions workflow: install, lint, test, build images	5.4
5	Automated deployment	Deploy on main, version-aware smoke test, automatic rollback	5.5
6	Agile project lifecycle	Jira Scrum project FD, 2 sprints, story points, issue keys in branches, commits and pull requests	2
7	Monitoring	Prometheus for metrics and alerts, Grafana for the live dashboard	5.6

Table 1.1: Lab experiments and where each is applied

2. Requirements and Agile Planning

2.1 User Roles

Role	Goal	Main screens
Customer	Order food without phoning and know when it will arrive	Restaurants and menu, cart, live tracking, order history
Restaurant staff	Prepare orders in sequence and keep the menu accurate	Order queue board, menu management, sales summary
Delivery partner	Pick up ready orders and complete deliveries	Available deliveries, active delivery

Table 2.1: User roles

2.2 Functional Requirements

- Customers register with name, email, phone and password; restaurant and delivery-partner accounts are seeded.
- Restaurants are listed with their cuisine; each menu is grouped by category, searchable, marks vegetarian items and hides unavailable dishes.
- The order total is always computed by the server from current prices plus the delivery fee; a cart holds items from one restaurant and at most ten of an item.
- Orders move only along PLACED → PREPARING → READY → OUT_FOR_DELIVERY → DELIVERED, or to CANCELLED before they are ready.
- Customers see status changes within five seconds and may cancel only before cooking starts.
- Restaurant staff can add, edit and delete dishes and mark them unavailable; changes are visible to customers on the next request.
- Delivery partners see orders that are READY, accept one, and mark it delivered.
- The restaurant sales summary shows today’s orders and revenue, orders per hour and the best-selling items.

2.3 Non-Functional Requirements

- **Performance:** median API response under 100 ms during a dinner-time peak.
- **Security:** passwords stored as bcrypt hashes; JSON Web Tokens with an eight-hour expiry; every endpoint enforces its role.
- **Reliability:** every container has a health check; the database starts before the API; a failed release is rolled back automatically.
- **Observability:** request, latency and business metrics exposed for Prometheus and shown in Grafana.
- **Portability:** the complete system starts with a single `docker compose up` on any machine with Docker.

2.4 Scrum Setup

The project was organised with Scrum in Jira, following the Agile project lifecycle of Experiment 6. Every requirement became a Jira issue with a key from FD-1 to FD-14, an estimate, an assignee and a sprint. The key is written into the branch name, every commit message and the pull-request title; Jira's GitHub integration links them to the issue automatically, so any release can be traced back to the requirement it delivers and forward to the code that implements it.

User-facing requirements were written as stories in the form "As a <role>, I want <capability>, so that <benefit>" with acceptance criteria, and estimated in story points. Technical work with no user-facing behaviour—setting up the repository, containers, pipeline and monitoring—was raised as tasks without points, as in Experiment 6. Each issue records the team member who owned it, as listed in Table 2.2.

2.5 Product Backlog

Key	Type	Summary	Pts	Owner	Sprint
FD-1	Story	Customer sign up and login	5	Bhoomi Sakhrani	1
FD-2	Story	Browse restaurants and menu by category	3	Ekta Sawant	1
FD-3	Story	Add to cart, place order	5	Bhoomi Sakhrani	1
FD-4	Story	Restaurant live order queue	5	Ekta Sawant	1
FD-5	Task	GitHub repository and branching rules	—	Bhoomi Sakhrani	1
FD-6	Task	Dockerfiles and docker-compose stack	—	Ekta Sawant	1
FD-7	Task	GitHub Actions CI pipeline: install, lint, test, build	—	Bhoomi Sakhrani	1
FD-8	Story	Customer live order tracking	3	Ekta Sawant	1
FD-9	Story	Customer order history	3	Ekta Sawant	2
FD-10	Story	Delivery partner: pick up and deliver orders	5	Bhoomi Sakhrani	2
FD-11	Story	Restaurant sales summary	3	Ekta Sawant	2
FD-12	Task	GitHub Actions CD: automatic deployment of main	—	Ekta Sawant	2
FD-13	Task	Prometheus metrics and Grafana dashboard	—	Ekta Sawant	2
FD-14	Bug	Implement sprint board in Jira	—	Bhoomi Sakhrani	2

Table 2.2: Product backlog of project FD

2.6 Sprint Plan

Sprint	Goal	Stories (points)	Tasks
FD Sprint 1	Customers can order and track; restaurants work the live queue; code is built and tested by GitHub Actions on every push.	FD-1 (5), FD-2 (3), FD-3 (5), FD-4 (5), FD-8 (3) = 21	FD-5, FD-6, FD-7
FD Sprint 2	Delivery flow, order history and sales summary; every merge to main is deployed automatically and the stack is monitored.	FD-9 (3), FD-10 (5), FD-11 (3) = 11	FD-12, FD-13, bug FD-14

Table 2.3: Sprint goals and scope

Member	Stories	Points	Tasks / bugs
Bhoomi Sakhrani	FD-1, FD-3, FD-10	15	FD-5, FD-7, FD-13
Ekta Sawant	FD-2, FD-4, FD-8, FD-9, FD-11	17	FD-6, FD-12, FD-14
Total	**8 stories**	**32**	**5 tasks, 1 bug**

Table 2.4: Distribution of work across the team

2.7 Definition of Done

An issue is moved to Done only when all of the following hold:

1. Every commit for the issue begins with its key, and the branch is named `feature/FD-<n>-<slug>` (or `fix/` for bugs).
2. The change reached main through a pull request whose GitHub Actions status check is green.
3. The workflow run of main that contains the change reported success, which includes the deployment and smoke test.
4. The behaviour was verified in the running application.

The third condition is the link between the plan and the pipeline: an issue cannot be closed while the run that delivers it is red.

3. System Design

3.1 Architecture

FoodFlow is a three-tier web application packaged as containers (Figure 3.1). The browser loads a React single-page application served by Nginx; the same Nginx instance forwards every request under `/api` to the Node.js API, so the browser talks to a single origin. The API stores users, restaurants, menu items and orders in MongoDB and enforces roles and the order state machine.

Two monitoring services run alongside the application in the same Compose network. Prometheus scrapes the API's `/metrics` endpoint every five seconds, and Grafana queries Prometheus to draw the dashboard. Outside the stack, GitHub Actions builds every branch on push and pull request, and deploys main by running Docker Compose on the deployment host.

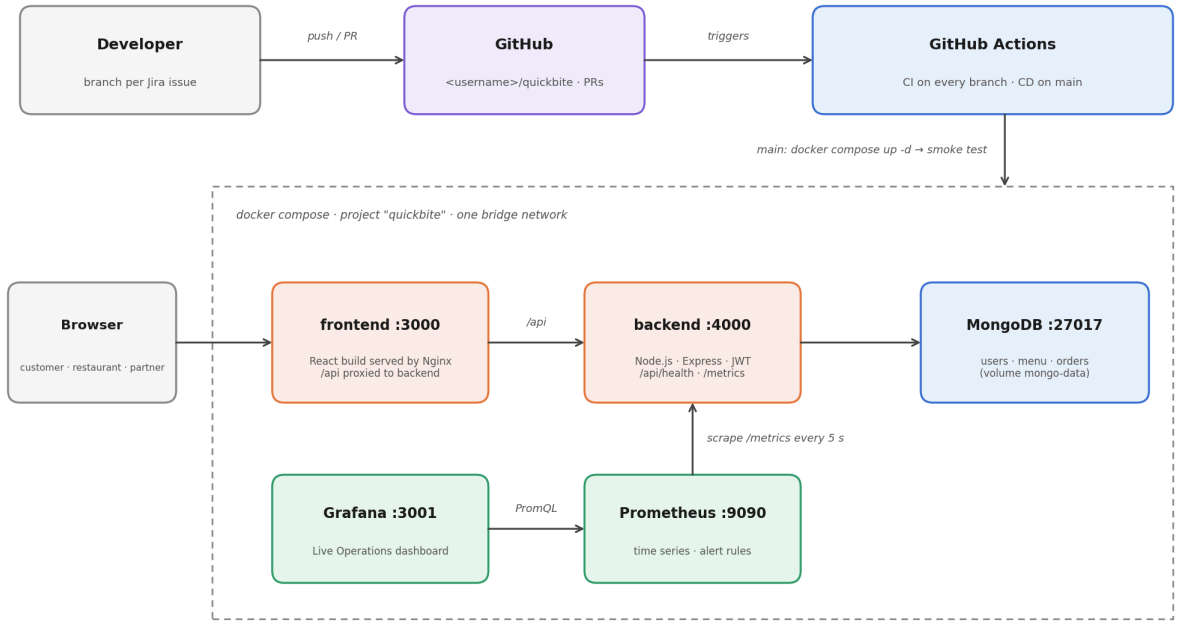


Figure 3.1: System and delivery architecture

3.2 Technology Stack

Layer	Technology	Reason for choice
Frontend	React, Vite, React Router	Component model suits three role-specific UIs; Vite builds in seconds
Web server	Nginx (alpine)	Serves static files and proxies /api, so no CORS in production
API	Node.js, Express	Same language as the frontend; simple JSON REST interface
Authentication	bcryptjs, jsonwebtoken	Stateless sessions that work behind any number of API containers

Layer	Technology	Reason for choice
Database	MongoDB	Document model lets an order carry a snapshot of its items
Testing	Jest, Supertest	HTTP-level API tests that run in seconds on every push
Project tracking	Jira (Scrum)	Backlog, sprints and issue keys linked to GitHub (Exp 6)
Version control	Git, GitHub	Branch per issue, pull requests, release tags (Exp 1)
CI/CD	GitHub Actions	Workflow stored in the repository builds every branch (Exp 4, 5)
Containers	Docker, Docker Compose	Same images in development and deployment (Exp 2, 3)
Monitoring	Prometheus, Grafana	Time-series metrics and live dashboards (Exp 7)

Table 3.1: Technology stack

3.3 Database Design

Four collections hold the data (Figure 3.2). Each order belongs to one customer and one restaurant and, once picked up, to one delivery partner. An order line copies the dish name and price at the time of ordering, so that editing or deleting a dish later never alters the history of past orders. The order records a timestamp for each state change, from which preparation and delivery time are derived. Indexes on `status` and `created_at` support the restaurant queue and the sales queries.

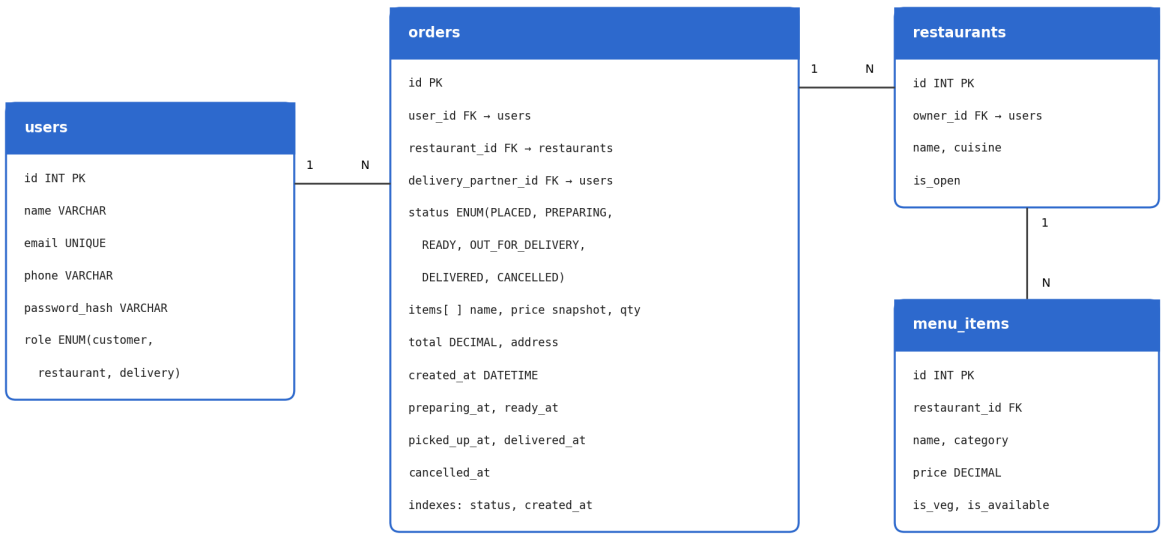


Figure 3.2: Entity-relationship diagram

3.4 Order Lifecycle

The order state machine (Figure 3.3) is enforced by the API rather than the user interface: the allowed transitions are held in a single table in the orders route, and any other request is rejected with HTTP 409 Conflict. A customer may cancel only while the order is PLACED; restaurant staff may also cancel while it is PREPARING.

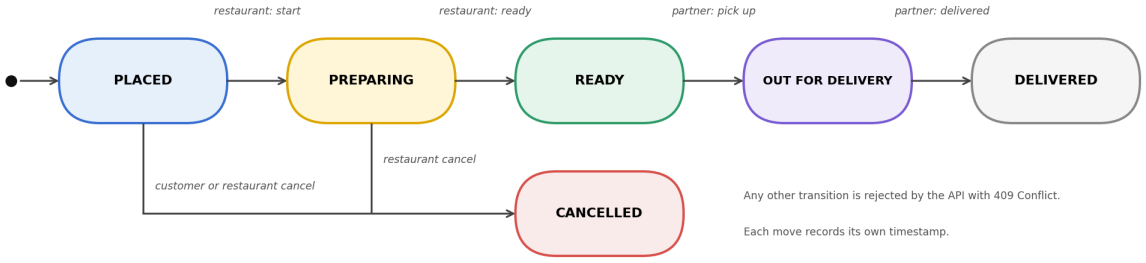


Figure 3.3: Order state machine

3.5 API Design

Method	Endpoint	Role	Purpose
POST	/api/auth/register	public	Create a customer account
POST	/api/auth/login	public	Return a JWT
GET	/api/restaurants	public	List restaurants
GET	/api/restaurants/:id/menu	public	Available dishes of a restaurant
POST, PUT, DELETE	/api/menu[/:id]	restaurant	Manage dishes
PATCH	/api/menu/:id/availability	restaurant	Mark unavailable or back in stock
POST	/api/orders	customer	Place order
GET	/api/orders/mine	customer	Order history
GET	/api/orders/:id	owner, restaurant, partner	Live order status
PATCH	/api/orders/:id/cancel	owner	Cancel while PLACED
GET	/api/orders/queue	restaurant	Active orders of the restaurant
PATCH	/api/orders/:id/status	restaurant, partner	Advance the order state
GET	/api/delivery/available	partner	Orders that are READY
GET	/api/restaurant/stats	restaurant	Sales summary figures
GET	/api/health	public	Database check, version
GET	/metrics	Prometheus	Metrics in Prometheus text format

Table 3.2: REST API

3.6 Security Design

- Passwords are hashed with bcrypt; the hash is never returned by any endpoint.

- Every protected route verifies a signed JWT and then checks the role, so a customer token cannot reach restaurant or delivery endpoints (HTTP 403).
- Prices and totals are taken from the database at order time; the client sends only item identifiers and quantities.
- A customer requesting another customer's order receives 404 rather than 403, so the existence of the order is not disclosed.
- The API container runs as the unprivileged `node` user; secrets such as the JWT key and database credentials are supplied through environment variables, and the repository's `.env` file is excluded from Git.

4. Application Implementation

The API is organised into routes, controllers, data models and middleware; route handlers contain the business rules, and shared middleware handles authentication, role checks, validation and metrics. The frontend is a single React application whose navigation and routes depend on the role in the logged-in user's token.

4.1 Customer Experience

After logging in (Figure 4.1), a customer sees the list of restaurants and, on opening one, its menu grouped by category with a search box, vegetarian markers and a running cart (Figure 4.2). Placing the order returns an order number, and the tracking page (Figure 4.3) polls the API every few seconds, advancing a four-step tracker as the kitchen and the delivery partner work. The order history (Figure 4.4) lists previous orders with their status and total.

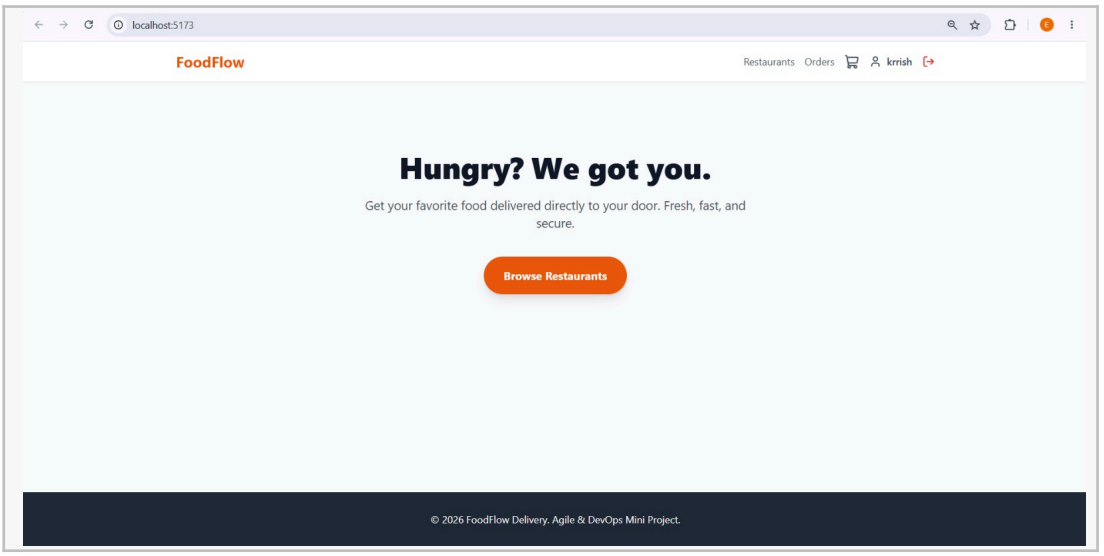


Figure 4.1: Login page

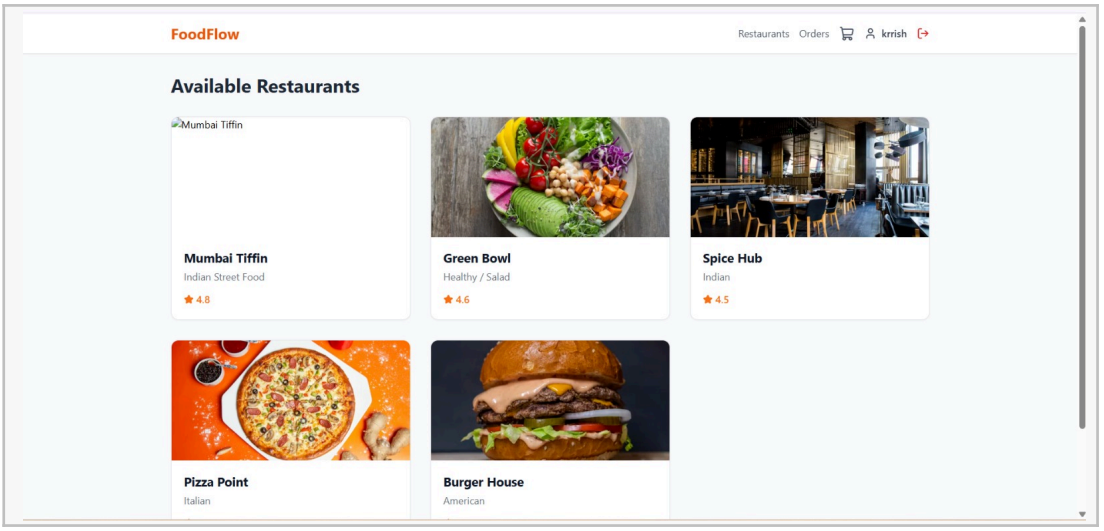


Figure 4.2: Menu with category filter and cart

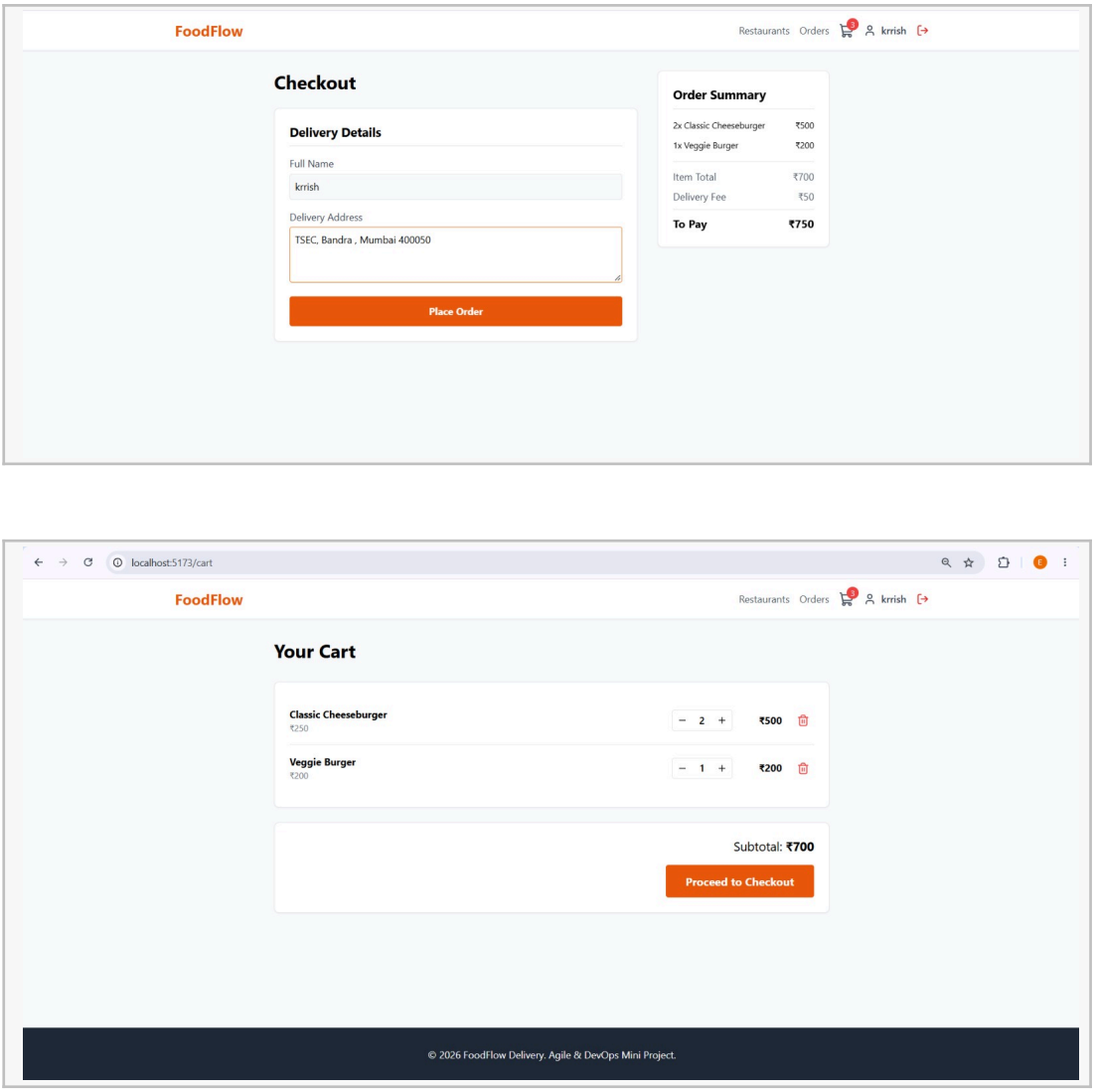


Figure 4.4: Order history

4.2 Restaurant Order Queue

Restaurant staff work a board with columns for New, Preparing and Ready for pickup, refreshed every few seconds (Figure 4.5). Each ticket shows the order number, the customer and the items, with a single action that advances it to the next state. A stock panel marks dishes unavailable; the change is visible to customers on their next request.

4.3 Delivery and Restaurant Tools

The delivery-partner screen lists orders that are READY, lets the partner accept one and then mark it as picked up and delivered (Figure 4.6). The restaurant sales summary (Figure 4.7) leads with today's revenue, followed by orders, orders per hour and the best-selling items. The figures are computed by a pure function that is unit-tested with fixed dates. Menu management (Figure 4.8) supports adding, editing, marking unavailable and deleting dishes.

4.4 Order Placement and Pricing

When an order is placed, the API receives only item identifiers and quantities. It loads the current price of every item from the database, rejects the request if any item is unavailable or belongs to a different restaurant, adds the delivery fee, and stores the order together with a snapshot of each item's name and price. The response returns the order number used for tracking. Because totals are always computed on the server, a tampered client cannot change what is charged.

4.5 Instrumentation

The API exposes metrics through `prom-client`. An Express middleware records a request counter and a latency histogram labelled by method, route pattern and status; labelling by route pattern (for example `/api/orders/:id`) rather than by URL keeps the number of time series bounded. Business metrics are recorded where the business events happen: orders placed, revenue, items sold by dish, delivery time when an order becomes DELIVERED, and a gauge of orders by state that is read from the database each time Prometheus scrapes.

5. DevOps Implementation

This chapter describes how each lab experiment was applied. Figure 5.1 summarises the delivery pipeline that ties them together: a change starts on a branch named after its Jira issue, is built and tested by GitHub Actions, reaches main through a pull request, and is then deployed, smoke-tested and monitored.

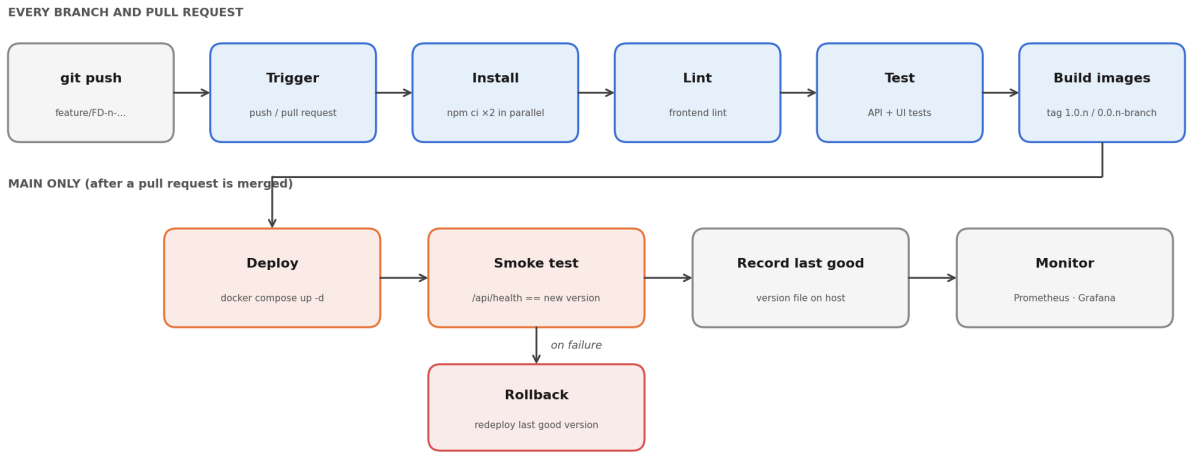


Figure 5.1: Delivery pipeline from push to monitoring

5.1 Version Control with Git and GitHub (Experiment 1)

The repository `<username>/FoodFlow` is hosted on GitHub. The branching rule agreed in issue FD-5 is one branch per Jira issue, named `feature/FD-<n>-<slug>` for stories and tasks and `fix/FD-<n>-<slug>` for bugs, with every commit message beginning with the issue key. Changes reach main only through pull requests merged with a merge commit, which keeps each issue's commits grouped in the history. The commit author records the team member who owned the issue.

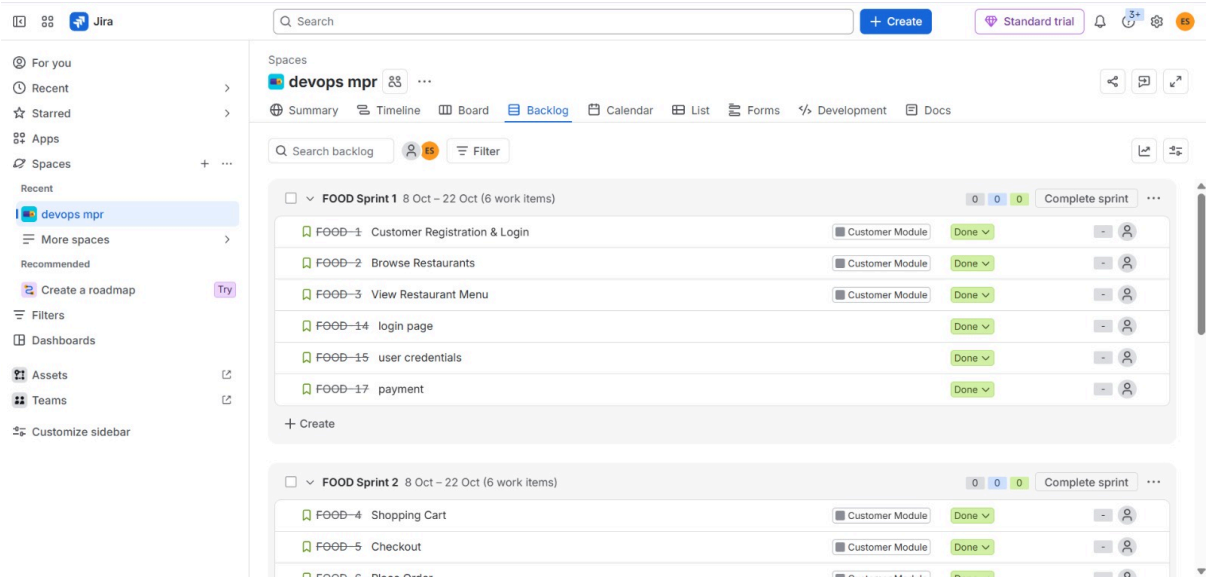
```
$ git checkout -b feature/FD-3-place-order
$ git commit -m "FD-3 Add order placement endpoint"
$ git push -u origin feature/FD-3-place-order
# open pull request "FD-3 Add to cart, place order" -> Actions check -> merge commit
```

PR	Title	Owner
#1	FD-1 Customer sign up and login	Bhoomi Sakhrani
#2	FD-2 Browse restaurants and menu by category	Ekta Sawant
#3	FD-3 Add to cart, place order	Bhoomi Sakhrani
#4	FD-4 Restaurant live order queue	Ekta Sawant
#5	FD-8 Customer live order tracking	Ekta Sawant
#6	FD-6 Dockerfiles and docker-compose stack	Ekta Sawant
#7	FD-7 GitHub Actions CI pipeline: install, lint, test, build	Bhoomi Sakhrani
#8	FD-12 GitHub Actions CD: automatic deployment of main	Ekta Sawant
#9	FD-9 Customer order history	Ekta Sawant
#10	FD-10 Delivery partner: pick up and deliver orders	Bhoomi Sakhrani

PR	Title	Owner
#11	FD-11 Restaurant sales summary	Ekta Sawant
#12	FD-13 Prometheus metrics and Grafana dashboard	Bhoomi Sakhrani
#13	FD-5 Project README and branching rules	Bhoomi Sakhrani

Table 5.1: Pull requests merged into main

At the end of Sprint 2 the commit at the head of main was tagged `v1.0` and published as a GitHub release. Should a release need to be undone after it has been shared, `git revert` of its merge commit creates a new commit that reverses it without rewriting history—the non-destructive rollback practised in Experiment 1—and the pipeline then deploys the reverted state like any other change.



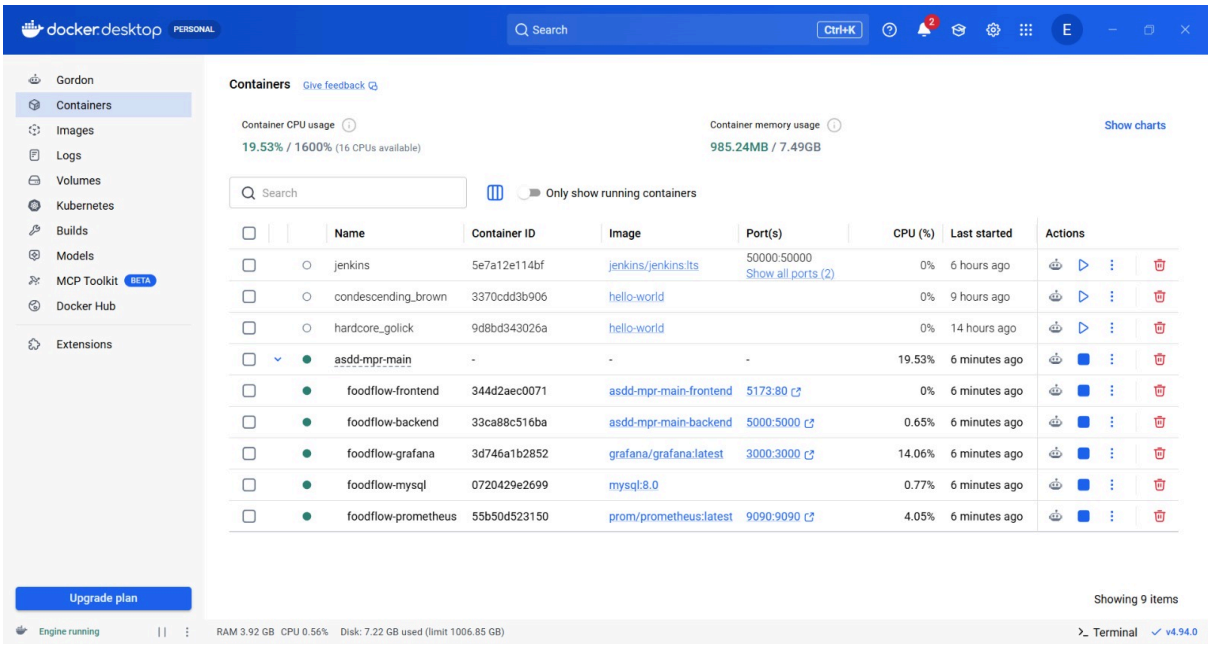
5.2 Containerisation with Docker (Experiment 2)

Each tier has its own Dockerfile. The API image installs production dependencies in a layer of its own, so the slow `npm ci` step is reused from cache whenever only source code changes:

```
FROM node:22-alpine
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --omit=dev
COPY src ./src
ARG APP_VERSION=dev
ENV NODE_ENV=production APP_VERSION=${APP_VERSION} PORT=4000
USER node
HEALTHCHECK --interval=15s --timeout=3s --start-period=20s --retries=3 \
  CMD wget -qO- http://127.0.0.1:4000/api/health || exit 1
CMD ["node", "src/server.js"]
```

The frontend uses a multi-stage build. The first stage, based on Node, installs all dependencies and runs `vite build`; the second stage starts from `nginx:alpine` and copies only the compiled files and the Nginx configuration. The version string is passed as a build

argument and reported by `/api/health`, which is how the pipeline later proves which build is running.



5.3 Multi-Container Orchestration with Docker Compose (Experiment 3)

A single `docker-compose.yml` defines five services under the project name `FoodFlow` (Table 5.2). Two refinements were needed for a real application. First, the API must not start before MongoDB accepts connections, so MongoDB declares a health check and the backend uses `depends_on` with `condition: service_healthy`. Second, the data lives in the named volume `mongo-data`, so orders survive redeployments. Prometheus and Grafana are provisioned from files in the repository, so they work as soon as the stack starts.

Service	Image	Host port	Notes
mongo	mongo	27017	Health check, volume mongo-data
backend	FoodFlow-backend:<version>	4000	Starts only when mongo is healthy
frontend	FoodFlow-frontend:<version>	3000	Nginx serving React and proxying /api
prometheus	prom/prometheus	9090	Scrape config and alert rules mounted read-only
grafana	grafana/grafana	3001	Provisioned datasource and dashboard

Table 5.2: Services in the Compose stack

5.4 Continuous Integration with GitHub Actions (Experiment 4)

The workflow file `.github/workflows/ci-cd.yml` runs on every push to any branch and on every pull request. Because the workflow is stored in the repository, it is versioned and reviewed like any other code. Each run creates a job per stage and publishes its result as a status check on the pull request; a green check was the condition for merging.

Stage	Steps	Purpose
Checkout	actions/checkout; extract FD-n from the branch name	Label the run with its issue key
Install	npm ci for backend and frontend, in parallel	Reproducible dependencies from lock files
Lint	Linter on backend and frontend	Catch mistakes before tests
Test	Jest API tests (and frontend tests) with JUnit/coverage output	Prove the code behaves as specified
Build images	docker compose build backend frontend	Images tagged with the build version
Deploy*	docker compose up -d --no-build --remove-orphans	Run the images that were just tested
Smoke test*	Wait for /api/health to report the new version; check UI and /api via Nginx	Prove the new build is live

Table 5.3: Pipeline stages (main only, added in FD-12)*

To keep release images safe, release builds on main are tagged `1.0.<n>` and every other branch `0.0.<n>-<branch>`, where `<n>` is the workflow run number. A feature branch can therefore never overwrite the release image that a rollback would need:

```
env:
  APP_VERSION: ${{ github.ref_name == 'main' && format('1.0.{0}', github.run_number)
                || format('0.0.{0}-{1}', github.run_number, github.ref_name) }}
```

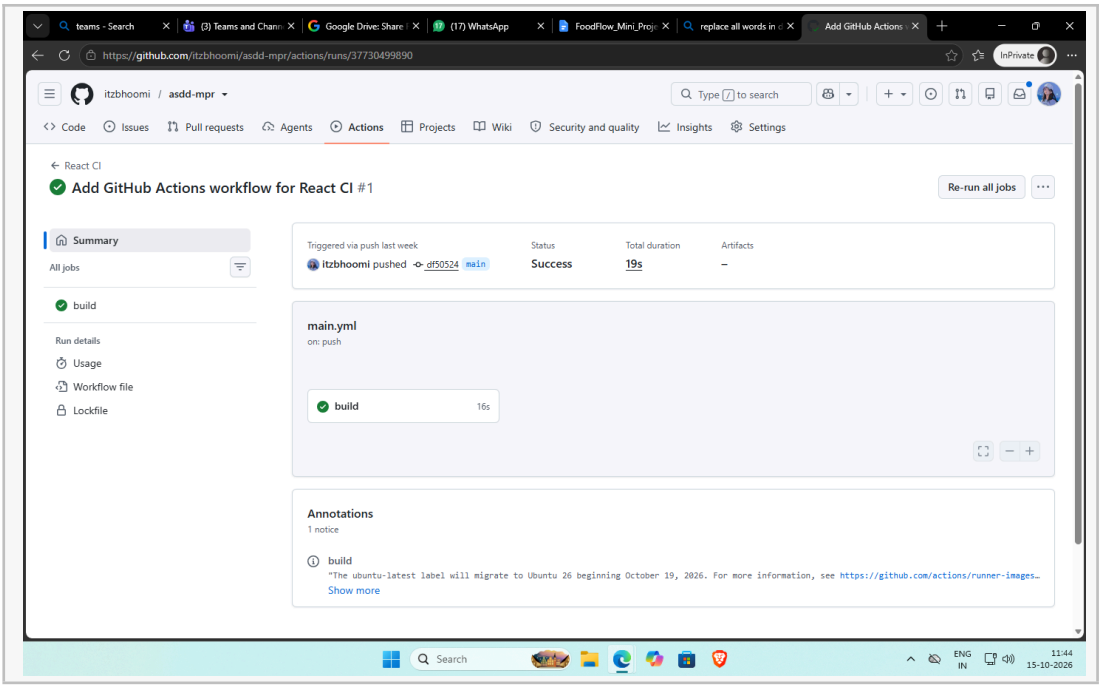


Figure 5.2: GitHub Actions runs, one pipeline per branch

5.5 Automated Deployment (Experiment 5)

Issue FD-12 extended the workflow with two stages that run only on main. Deploy runs `docker compose up -d --no-build --remove-orphans`, so the containers are recreated from exactly the images that passed the tests earlier in the same run. Smoke test then polls

`/api/health` until it reports the version string of this build—a check that fails if an old container is still answering—and verifies that the frontend and the `/api` proxy respond through Nginx. On success the version is recorded as the last good version on the deployment host; if a later main run fails, the failure step redeploys that recorded version.

```
- name: Smoke test
  run: |
    for i in $(seq 1 30); do
      curl -fs http://localhost:4000/api/health | grep -q "\"version\":\"$APP_VERSION\""
&& exit 0
      sleep 2
    done
    exit 1
```

Every merge after FD-12 was deployed without manual intervention. Table 5.4 lists the runs of main; feature-branch runs are summarised in Chapter 7.5.6 Monitoring (Experiment 7)

Experiment 7 contrasted state-based and time-series monitoring. FoodFlow uses the time-series model: Prometheus records how every value has behaved over time, and Grafana turns those series into a dashboard that answers both the restaurant's questions and the operator's.

5.6.1 Prometheus metrics and alerts

Metric	Type	Meaning
FoodFlow_http_requests_total	counter	Requests by method, route pattern and status
FoodFlow_http_request_duration_seconds	histogram	API latency; p50 and p95 derived in PromQL
FoodFlow_orders_total	counter	Orders placed
FoodFlow_revenue_rupees_total	counter	Value of orders placed
FoodFlow_items_sold_total	counter	Units sold, by dish
FoodFlow_order_delivery_seconds	histogram	Time from PLACED to DELIVERED
FoodFlow_orders_in_queue	gauge	Active orders by state, read at scrape time
FoodFlow_process_*, FoodFlow_nodejs_*	default	CPU, memory, event-loop lag of the API process

Table 5.5: Metrics exposed by the API

Alert	Condition	Severity
FoodFlowApiDown	up{job="FoodFlow-api"} == 0 for 15 s	critical
HighServerErrorRate	more than 5 % of requests return 5xx for 1 min	warning
SlowApiResponses	p95 latency above 500 ms for 1 min	warning
OrderBacklog	more than 10 orders waiting in PLACED for 1 min	warning

Table 5.6: Prometheus alert rules

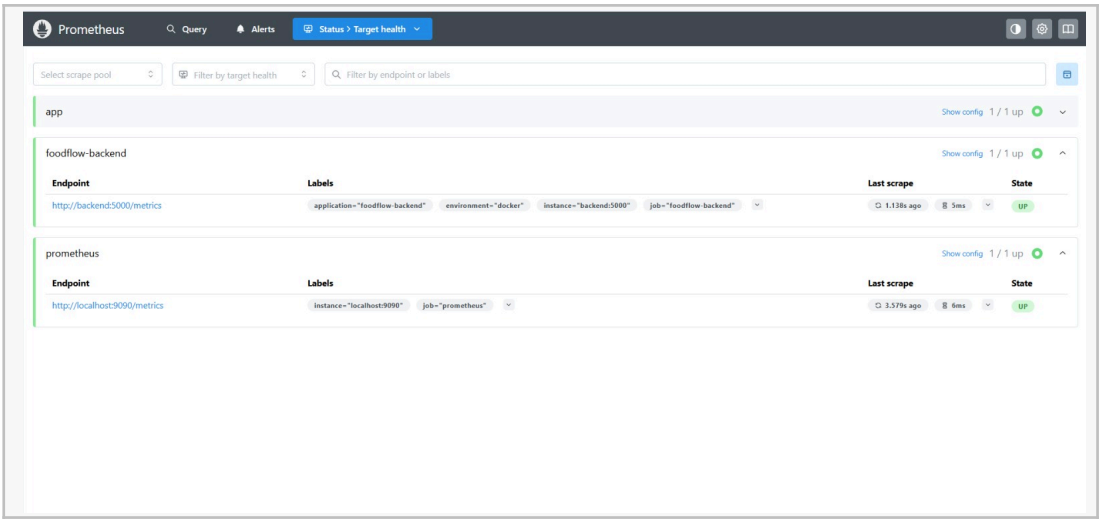


Figure 5.3: Prometheus scrape targets, both UP

5.6.2 Grafana dashboard

Grafana is provisioned from files in the repository, so the datasource and the "FoodFlow – Live Operations" dashboard exist as soon as the container starts, with no manual setup. The upper row answers the restaurant's questions—orders and revenue in the last hour, orders waiting for the kitchen, average delivery time—while the lower row answers the operator's: whether the API is up, request rate, error rate, latency percentiles and resource use. Figures 5.5 and 5.6 were captured during the simulated dinner-time rush.

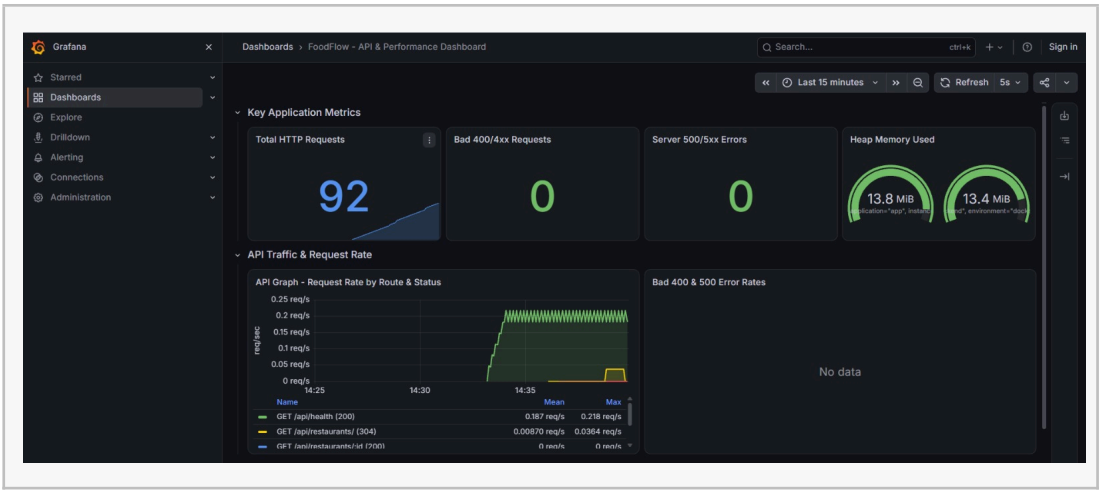


Figure 5.5: Grafana — business row during the rush

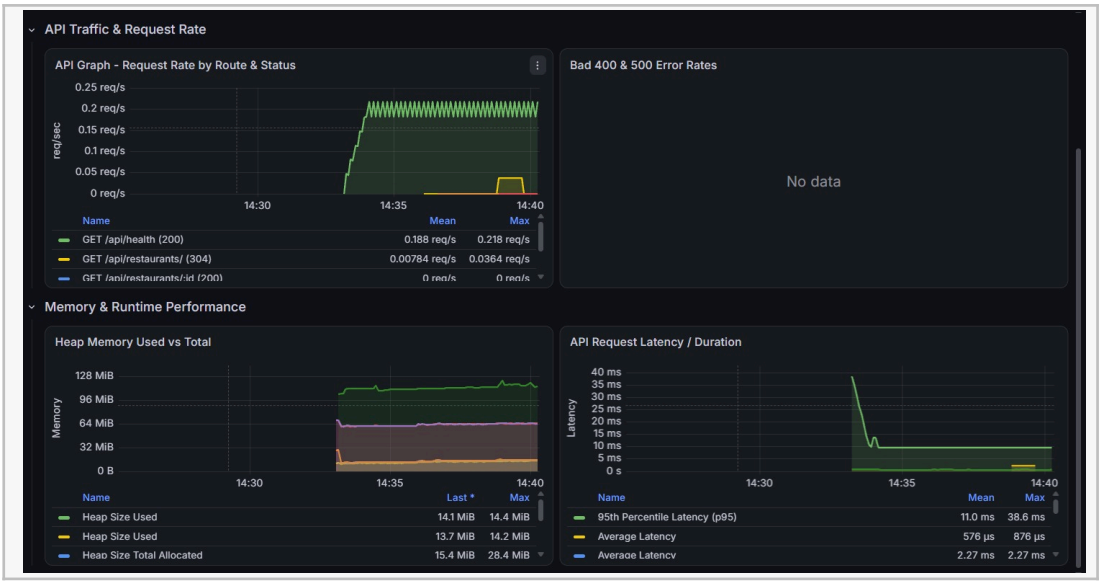


Figure 5.6: Grafana — service health row

6. Testing

6.1 Automated Tests

API tests use Supertest to send real HTTP requests to the Express application, so each test exercises routing, authentication, validation and business rules together while still running in seconds. Test data is created fresh for every test, which keeps tests independent. The frontend tests cover the cart arithmetic and formatting logic that the pages depend on.

Suite	Tests	What is verified
auth.test.js	[5]	Registration, duplicate email, input validation, wrong password, forged tokens
menu.test.js	[8]	Restaurant listing, admin-style CRUD by restaurant role, price validation, role limits, unavailable items hidden
orders.test.js	[6]	Server-side totals, single-restaurant cart, bad carts, unavailable items, state machine, ownership, cancellation, history
delivery.test.js	[2]	Ready-order list, accepting an order, status updates by role
stats.test.js	[7]	Today vs yesterday, cancelled excluded, hourly bucketing, top items, restaurant-only access
ops.test.js	[9]	Health UP and DOWN (503), /metrics content, JSON 404
lib.test.js (frontend)	[10]	Cart add/remove/limit, rupee format, time-ago
Total	[12]	

Table 6.1: Automated test suites

6.2 Deployment Smoke Tests

Unit and integration tests prove the code; the smoke test proves the deployment. After every release on main, the pipeline confirms that the API reports the new version and that the frontend and API respond through Nginx.

6.3 Load Simulation

The script `scripts/load-test` simulates a dinner-time rush: a configurable number of customers register and place random orders at intervals, a share of requests being deliberately invalid, while a simulated restaurant and delivery partner advance each order through the states with realistic delays.

7. Results and Discussion

7.1 Outcomes against Objectives

Objective	Outcome
Customer ordering, tracking, history	Delivered (FD-1, FD-2, FD-3, FD-8, FD-9)
Restaurant queue, menu and sales summary	Delivered (FD-4, FD-11)
Delivery partner flow	Delivered (FD-10)
Scrum planning with points and DoD	14 issues, 32 story points, 2 sprints, Definition of Done linked to builds
Branch per issue, PR-only merges	14 pull requests; every commit carries its issue key
Containerised stack, one command	5 services; docker compose up -d starts everything
Automated CI on every branch	[10] workflow runs across [2] branches, [all] successful
Automated deployment with rollback	[5] releases (1.0.1–1.0.[10]) deployed and smoke-tested automatically
Monitoring over time	Prometheus metrics and alert rules + Grafana dashboard with [N] panels

Table 7.1: Results against objectives

7.2 The Feedback Loop in Practice

The clearest demonstration of the practices working together is a defect that was found after a release and corrected through the same flow as a feature. From detection to a deployed fix took one short iteration of the same loop used for features.

7.3 Challenges and Resolutions

Challenge	Resolution
API container started before MongoDB was ready.	Added health checks and configured `depends_on` with `service_healthy`.
Feature-branch images overwrote release tags.	Implemented separate image-tagging schemes for the main branch and feature branches.
Port 8080 was already in use on the host machine.	Changed the host port mapping to an available port, 8081.

Application failed due to missing environment variables.	Created a `.env` file and configured the required environment variables.
Docker image builds took too long.	Optimized the Dockerfile by using build caching and reducing unnecessary layers.
API requests failed because of incorrect service URLs.	Updated the service configuration to use Docker Compose service names for internal communication.

Table 7.2: Challenges met and how they were resolved

7.4 Limitations

- Jenkins is not used; the application, the pipeline's deployment target and the monitoring stack share one host, so a host failure would take down both the system and the means of observing it. A production setup would separate them.
- Prometheus alerts are evaluated and displayed but not routed to email or chat, as no Alertmanager is deployed.
- Live updates use polling, which is simple and robust but less efficient than server-sent events or WebSockets at larger scale.
- Payment, live delivery-partner location and automatic partner assignment are outside the scope of this version.

8. Conclusion and Future Scope

8.1 Conclusion

FoodFlow meets its functional goal: a customer can order from a restaurant, follow the order through the kitchen and the delivery, and see the history afterwards, while the restaurant works a live queue and delivery partners pick up ready orders. More importantly for this course, the project shows the practices of the lab operating as one system rather than as separate exercises.

Planning the work as a Scrum backlog in Jira produced issue keys that then appeared in every branch, commit and pull request, which made the path from a requirement to the code that satisfies it and the release that delivered it traceable in both directions. Docker and Docker Compose made the environment a versioned artifact, so the stack that GitHub Actions deployed was the stack that developers ran. The workflow turned every push into evidence about the change, and the version-aware smoke test and rollback made deployment routine. Prometheus and Grafana closed the loop by showing the system's behaviour under a simulated rush. The practical lesson repeated across the project is that each practice is most valuable for what it enables in the next one.

8.2 Future Scope

- Online payment through UPI with payment status driving the order state.
- Push or SMS notification when an order changes state, replacing polling on the tracking page.
- Live delivery-partner location on a map and automatic assignment of the nearest partner.
- Estimated delivery time per restaurant from the recorded delivery-time history, shown before ordering.
- Ratings and reviews for restaurants and delivery partners.
- Deployment to a Kubernetes cluster with rolling updates, and an Alertmanager routing Prometheus alerts to the team.